Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет инфокоммуникационных технологий
Направление подготовки 09.03.03 Прикладная информатика

ОТЧЁТ

по лабораторной работе № 2
«Простой сайт на Django»

Дисциплина: Web-программирование
Вариант 2. Доска домашних заданий

Выполнил: Ашур Амир Куссаевич
Поток: WEB 2.1
Группа: K3339
Проверил: _____

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Введение 3
1 Практическая работа 2.1. Установка Django и первое приложение 4
1.1 Установка Django 4
1.2 Модель данных об автовладельцах 4
1.3 Админ-панель 5
1.4 Представление и маршрутизация 5
2 Практическая работа 2.2. Связь M2M, представления и формы 7
2.1 Связь «многие ко многим» 7
2.2 Вывод данных 7
2.3 Формы 8
3 Практическая работа 2.3. Расширение модели пользователя 9
4 Лабораторная работа. Вариант 2 «Доска домашних заданий» 11
4.1 Модель данных 11
4.2 Разграничение прав 12
4.3 Маршруты 13
4.4 Регистрация и вход 13
4.5 Список домашних заданий 14
4.6 Сдача работы 16
4.7 Журнал оценок 18
4.8 Интерфейсы преподавателя 19
4.9 Django Admin 22
4.10 Тестирование 22
4.11 Запуск проекта 23
Заключение 24

Введение

Цель работы: овладеть практическими навыками и умениями реализации web-сервисов средствами Django.

Задание: реализовать сайт на Django по шаблону MVC с использованием шаблонизатора и СУБД PostgreSQL согласно варианту. Предварительно выполнить практические работы 2.1–2.3.

Вариант 2 — «Доска домашних заданий». Хранится информация о предмете, преподавателе, дате выдачи, периоде выполнения, тексте задания и штрафах. Необходимо реализовать:

- регистрацию новых пользователей;
- просмотр домашних заданий по всем предметам (сроки выполнения, описания);
- сдачу домашних заданий в текстовом виде;
- выставление оценок преподавателем (администратором) средствами Django Admin;
- таблицу оценок всех студентов группы в клиентской части.

Для максимальной оценки дополнительно реализованы стилизация интерфейса (Bootstrap), пагинация, поиск и фильтрация объектов, а также разграничение прав студента и преподавателя в клиентском интерфейсе.

Таблица 1 — Используемые технологии

Компонент	**Версия / назначение**
Python	3.11
Django	5.2 — веб-фреймворк
PostgreSQL	16 — СУБД основного проекта
SQLite	СУБД практических работ (по методичке)
Bootstrap, Bootstrap Icons	5.3 / 1.11 — оформление интерфейса, подключены локально
MkDocs + Material	веб-версия отчёта

1 Практическая работа 2.1. Установка Django и первое приложение

1.1 Установка Django

Создано виртуальное окружение, установлен Django, созданы проект `django_project_ashur` и приложение `project_first_app`:

```
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install django
django-admin startproject django_project_ashur
cd django_project_ashur
python manage.py startapp project_first_app
```

Приложение добавлено в список `INSTALLED_APPS` файла `settings.py`.

1.2 Модель данных об автовладельцах

По схеме из методических указаний описаны четыре сущности: «Автовладелец», «Автомобиль», «Владение» (ассоциативная сущность с датами начала и окончания) и «Водительское удостоверение». Схема модели приведена на рисунке 1.

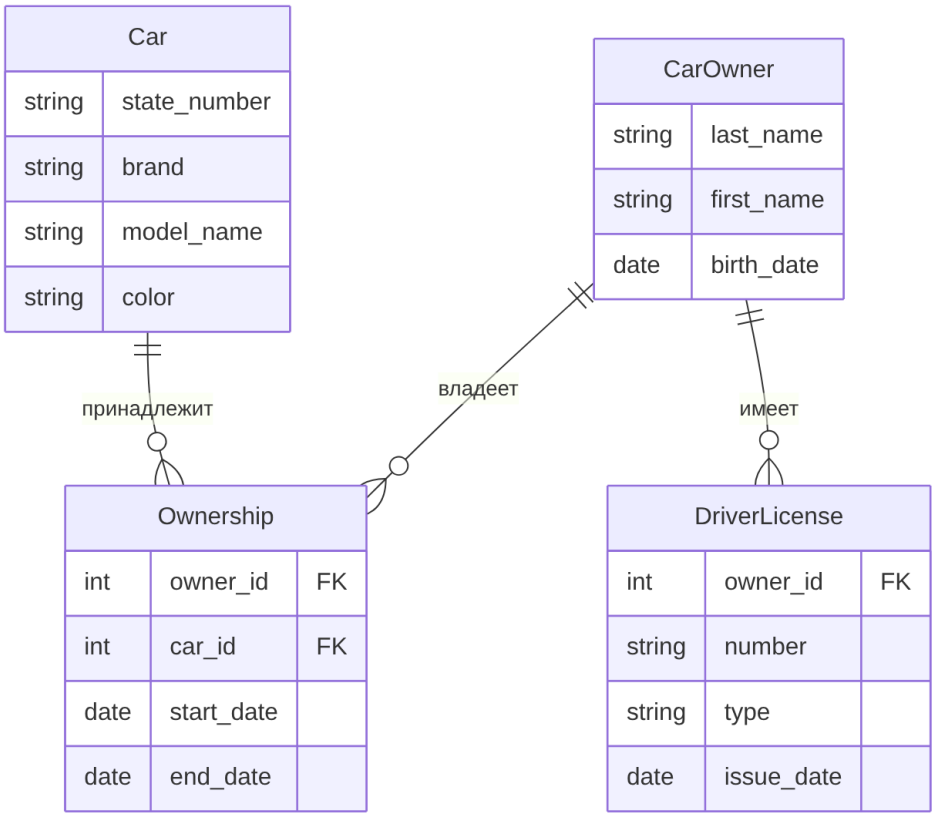


Рисунок 1 — Схема модели данных практических работ

```
class Car(models.Model):
    state_number = models.CharField('Гос. номер', max_length=15)
    brand = models.CharField('Марка', max_length=20)
    model_name = models.CharField('Модель', max_length=20)
    color = models.CharField('Цвет', max_length=30, null=True, blank=True)
    owners = models.ManyToManyField(settings.AUTH_USER_MODEL,
                                    through='Ownership')



class Ownership(models.Model):
    owner = models.ForeignKey(settings.AUTH_USER_MODEL,
                              on_delete=models.CASCADE)
    car = models.ForeignKey(Car, on_delete=models.CASCADE)
    start_date = models.DateField('Дата начала')
    end_date = models.DateField('Дата окончания', null=True, blank=True)



class DriverLicense(models.Model):
    TYPES = (('A', 'A — мотоциклы'), ('B', 'B — легковые'),
             ('C', 'C — грузовые'), ('D', 'D — автобусы'),
             ('M', 'M — мопеды'))
    owner = models.ForeignKey(settings.AUTH_USER_MODEL,
                              on_delete=models.CASCADE)
    number = models.CharField('Номер удостоверения', max_length=10)
    type = models.CharField('Тип', max_length=10, choices=TYPES)
    issue_date = models.DateField('Дата выдачи')
```

Модель «Автовладелец» в итоговой версии наследуется от `AbstractUser` (практическая работа 2.3). Это сделано сразу, так как параметр `AUTH_USER_MODEL` необходимо задать до первой миграции. Миграции выполнены командами `python manage.py makemigrations` и `python manage.py migrate`.

1.3 Админ-панель

Все модели зарегистрированы в `admin.py`, создан суперпользователь командой `python manage.py createsuperuser`. Добавлены владельцы и четыре автомобиля. Каждый из двух первых владельцев связан как минимум с тремя автомобилями, при этом периоды владения не пересекаются: автомобили переходили от одного владельца к другому. Тестовые данные загружаются скриптом `seed_practice.py`.

1.4 Представление и маршрутизация

Создано представление, выводящее данные владельца по его идентификатору, шаблон `owner.html` и маршрут с конвертером `int`:

```
# views.py
def owner_detail(request, owner_id):
    owner = get_object_or_404(CarOwner, pk=owner_id)
    return render(request, 'owner.html', {'owner': owner})

# project_first_app/urls.py
urlpatterns = [
    path('owner/<int:owner_id>/', views.owner_detail, name='owner_detail'),
]

# django_project_ashur/urls.py
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('project_first_app.urls')),
]
```

Результат по адресу `127.0.0.1:8000/owner/2/` показан на рисунке 2.

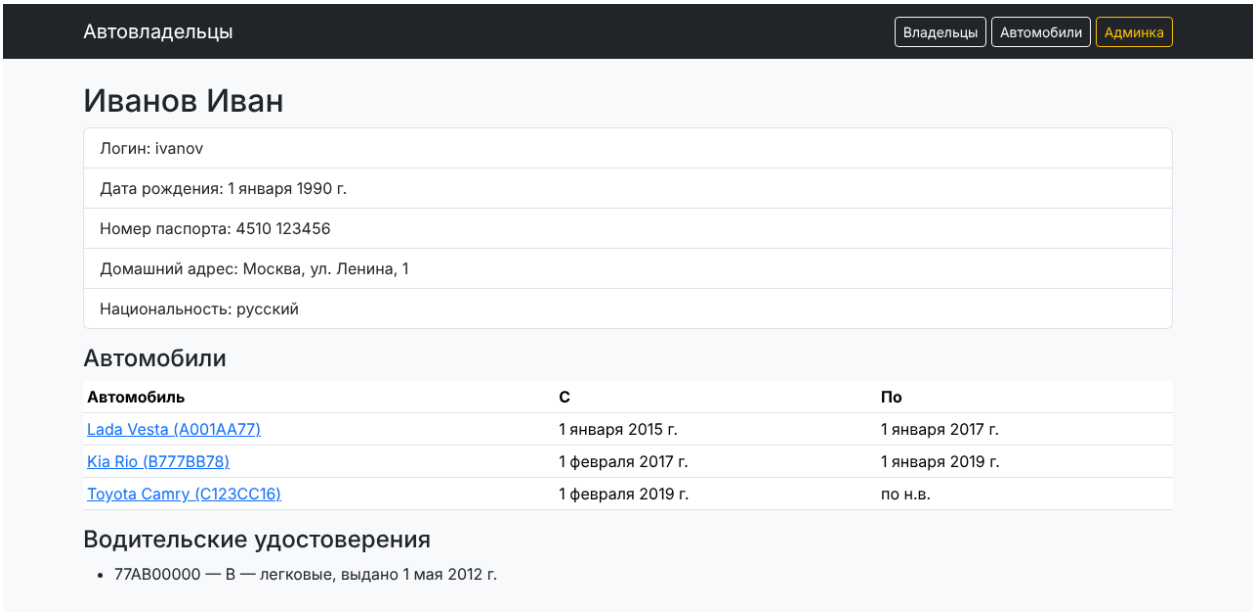


Рисунок 2 — Страница владельца автомобиля

2 Практическая работа 2.2. Связь M2M, представления и формы

2.1 Связь «многие ко многим»

Автомобиль связан с владельцем через ассоциативную сущность `Ownership` (параметр `through`). Так как у связи есть собственные атрибуты — даты начала и окончания владения, — записи создаются явно, а не методом `add()`:

```
Ownership.objects.create(owner=owner, car=car, start_date=date(2019, 2, 1))
```

2.2 Вывод данных

Список всех владельцев реализован функциональным представлением (FBV) по адресу `/owners/`:

```
def owner_list(request):
    owners = (CarOwner.objects.filter(is_superuser=False)
              .prefetch_related('ownerships__car'))
    return render(request, 'owner_list.html', {'owners': owners})
```

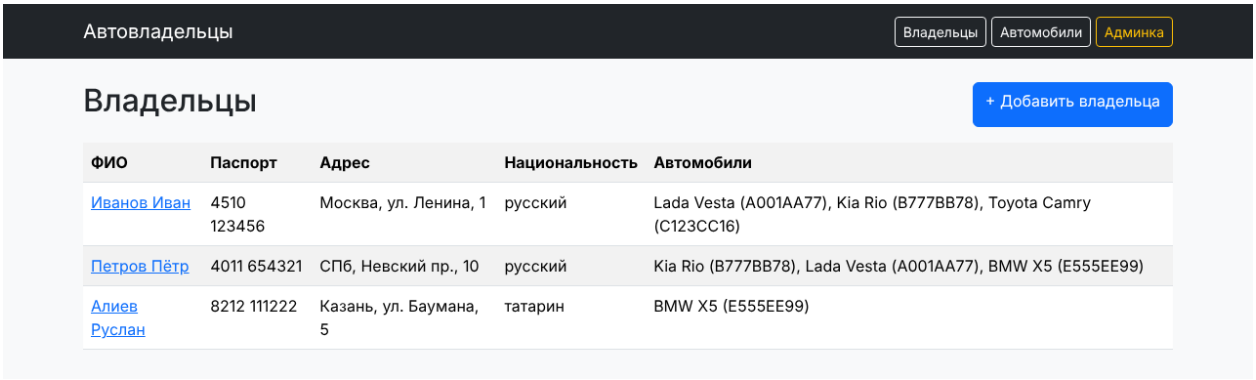


Рисунок 3 — Список владельцев (FBV)

Для автомобилей использованы классовые представления (CBV): `CarListView` — список, `CarDetailView` — просмотр по id, `CarUpdateView` — редактирование.

```
class CarListView(ListView):
    model = Car
    template_name = 'car_list.html'
    context_object_name = 'cars'


class CarDetailView(DetailView):
    model = Car
    template_name = 'car_detail.html'
```

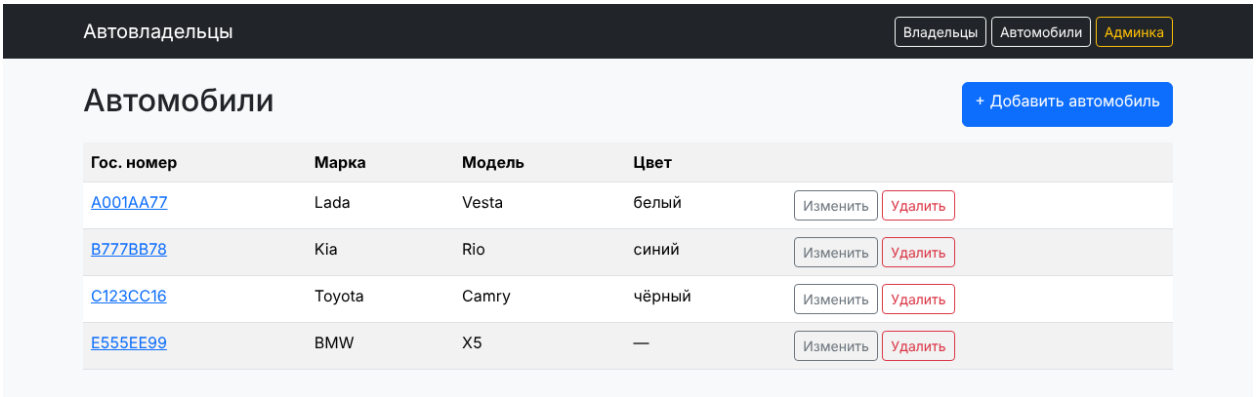


Рисунок 4 — Список автомобилей (CBV)

2.3 Формы

Форма ввода владельца реализована функциональным представлением. После успешной отправки выполняется перенаправление (`redirect`), чтобы при обновлении страницы не создавались дубликаты:

```
def owner_create(request):
    if request.method == 'POST':
        form = CarOwnerForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('owner_list')
    else:
        form = CarOwnerForm()
    return render(request, 'owner_form.html', {'form': form})
```

Создание, изменение и удаление автомобиля реализованы классами `CreateView`, `UpdateView` и `DeleteView`. Шаблон удаления содержит POST-форму с `{% csrf_token %}` и кнопкой подтверждения.

```
class CarCreateView(CreateView):
    model = Car
    form_class = CarForm
    template_name = 'car_form.html'
    success_url = reverse_lazy('car_list')


class CarDeleteView(DeleteView):
    model = Car
    template_name = 'car_confirm_delete.html'
    success_url = reverse_lazy('car_list')
```

3 Практическая работа 2.3. Расширение модели пользователя

Из четырёх способов расширения пользователя (proxy-модель, профиль через `OneToOneField`, `AbstractBaseUser`, `AbstractUser`) выбран `AbstractUser`: стандартная аутентификация подходит, а дополнительные поля нужны прямо в таблице пользователей. «Владелец автомобиля» стал пользователем и получил поля номера паспорта, домашнего адреса и национальности:

```
class CarOwner(AbstractUser):
    birth_date = models.DateField('Дата рождения', null=True, blank=True)
    passport_number = models.CharField('Номер паспорта', max_length=20,
                                       blank=True)
    home_address = models.CharField('Домашний адрес', max_length=255,
                                    blank=True)
    nationality = models.CharField('Национальность', max_length=50,
                                   blank=True)


# settings.py
AUTH_USER_MODEL = 'project_first_app.CarOwner'
```

Во всех внешних ключах используется `settings.AUTH_USER_MODEL`, а не прямой импорт класса пользователя. Новые поля выведены в Django Admin через наследника `UserAdmin` (рисунок 5):

```
@admin.register(CarOwner)
class CarOwnerAdmin(UserAdmin):
    fieldsets = UserAdmin.fieldsets + (
        ('Данные автовладельца', {'fields': (
            'birth_date', 'passport_number', 'home_address', 'nationality',
        )}),
    )
    inlines = [OwnershipInline, LicenseInline]
```

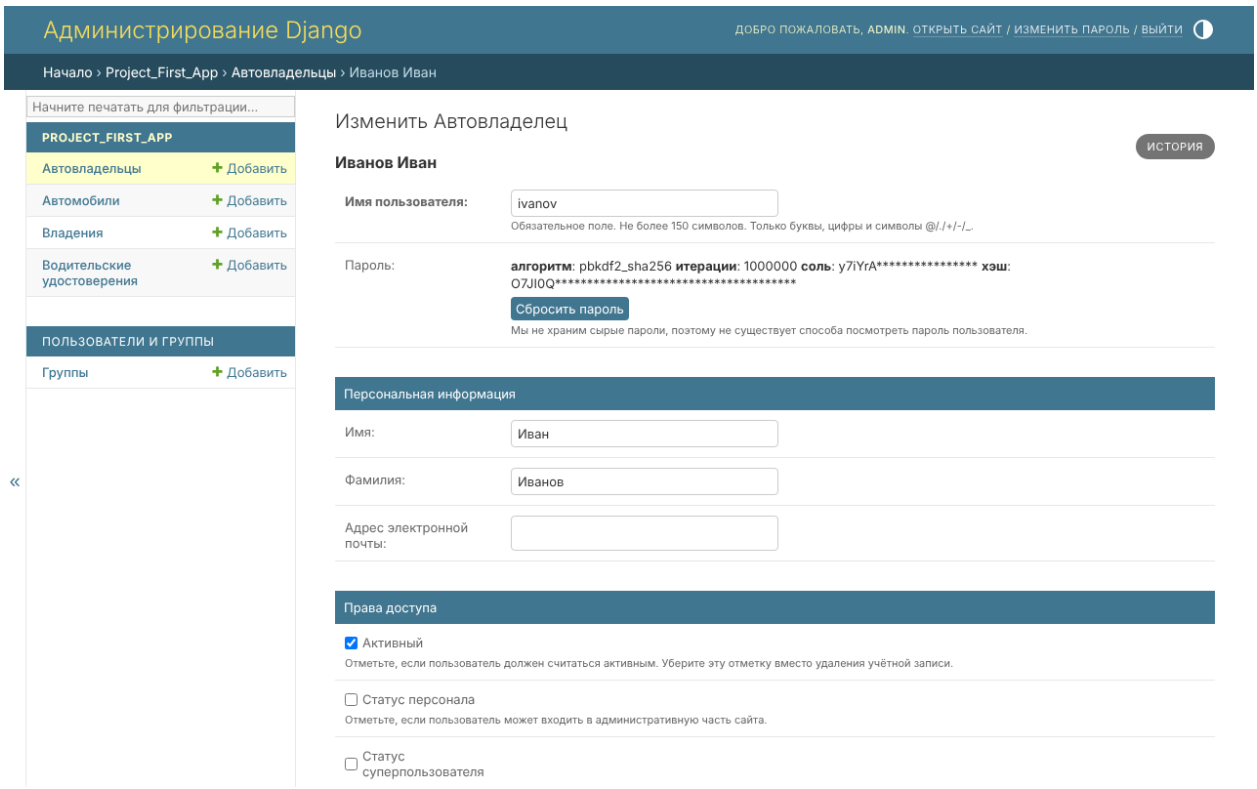


Рисунок 5 — Новые поля пользователя в Django Admin

Страницы владельца и списка владельцев из практических работ 2.1 и 2.2 доработаны: они выводят паспорт, адрес и национальность. Для создания пользователя с новыми атрибутами используется форма на основе `UserCreationForm`, пароль хэшируется стандартными средствами Django (рисунок 6).

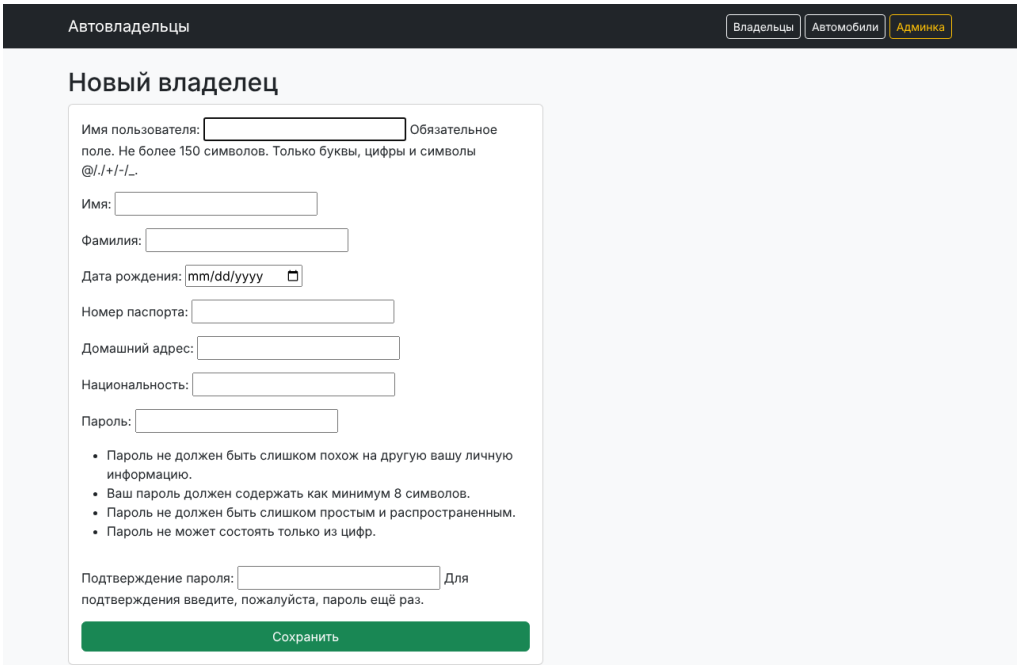


Рисунок 6 — Форма создания пользователя-автовладельца

4 Лабораторная работа. Вариант 2 «Доска домашних заданий»

4.1 Модель данных

Перед реализацией спроектирована структура базы данных (рисунок 7). Пользователь является либо студентом, либо преподавателем; роль определяется флагом `is_staff`. Одно задание может быть выдано нескольким группам, поэтому связь «задание — группа» сделана «многие ко многим».

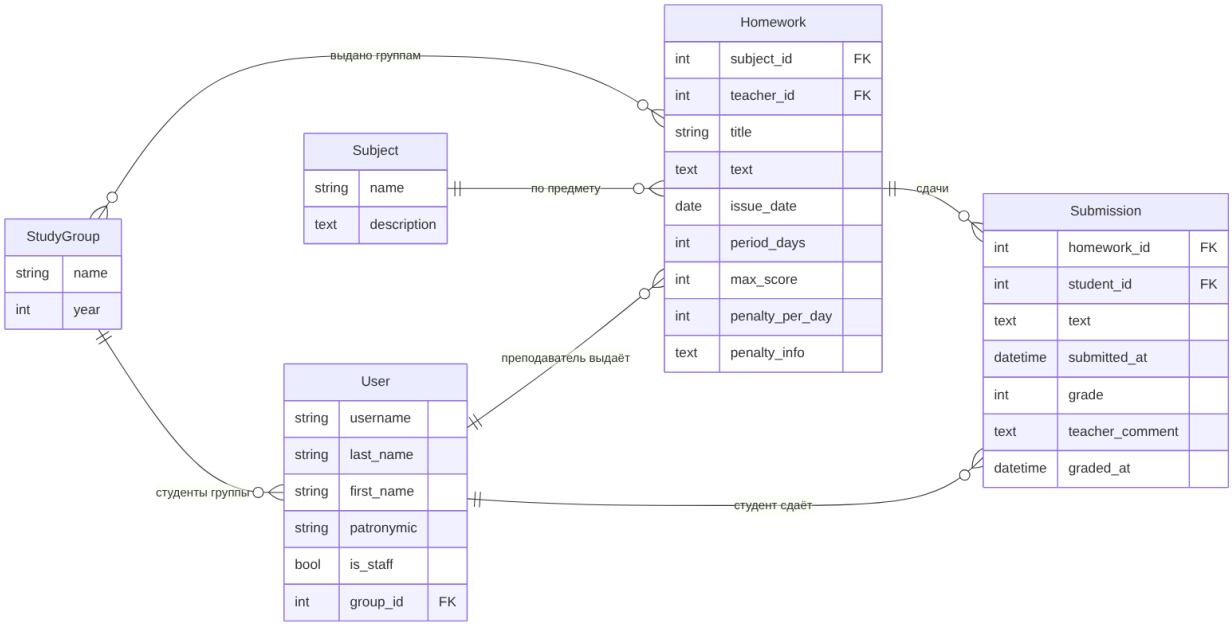


Рисунок 7 — Схема базы данных проекта

Таблица 2 — Модели проекта

Модель	**Назначение**	**Ключевые решения**
`User` (`AbstractUser`)	студент или преподаватель	роль задаёт флаг `is_staff`; у студента есть группа
`StudyGroup`	учебная группа	одно ДЗ выдаётся нескольким группам (M2M)
`Subject`	предмет	—
`Homework`	домашнее задание	дедлайн вычисляется: `issue_date + period_days`
`Submission`	сдача работы и оценка	одна сдача на студента и задание (`UniqueConstraint`); дата проверки ставится автоматически

Штраф хранится двумя полями: текстовым описанием (`penalty_info`) и числом баллов за день просрочки (`penalty_per_day`). По ним вычисляется рекомендуемая оценка, которую видит преподаватель:

```
@property
def days_late(self):
    return max(0, (timezone.localdate(self.submitted_at)
                   - self.homework.deadline).days)

@property
def suggested_grade(self):
    return max(0, self.homework.max_score
                  - self.days_late * self.homework.penalty_per_day)
```

Оценка проверяется на диапазон от 0 до максимального балла и в форме, и в методе `Model.clean()`, поэтому поставить балл выше максимума нельзя ни на сайте, ни в админке.

4.2 Разграничение прав

Таблица 3 — Возможности ролей

Возможность	**Гость**	**Студент**	**Преподаватель**
Регистрация и вход	+	—	—
Список ДЗ, поиск и фильтры	—	своя группа	все группы
Сдать, изменить, отменить свою работу до проверки	—	+	—
Журнал оценок	—	своя группа	любая группа
Выдать, изменить, удалить ДЗ	—	—	+
Очередь проверки и выставление оценок на сайте	—	—	+
Django Admin	—	—	+

Права проверяются на сервере (`LoginRequiredMixin`, `UserPassesTestMixin`, декоратор `user_passes_test`), а в шаблонах через `{% if user.is_staff %}` выводятся разные меню, кнопки и блоки. Чтобы преподаватель, не являющийся суперпользователем, мог выставлять оценки в Django Admin, миграция `0002_teachers_group` создаёт группу «Преподаватели» с правами на задания, сдачи и предметы. Сигнал `post_save` автоматически добавляет в неё каждого пользователя с `is_staff`.

Студент выбирает группу при регистрации, а сменить её в профиле не может: это делает только администратор. Иначе студент мог бы открыть журнал чужой группы.

4.3 Маршруты

Таблица 4 — Маршруты приложения

URL	Представление	Доступ
`/accounts/login/`, `/signup/`	`LoginView`, `signup`	все
`/`	`HomeworkListView` (CBV, 6 на странице)	авторизованные
`/homework/<pk>/`	`homework_detail`	авторизованные
`/homework/new/`, `…/edit/`, `…/delete/`	`CreateView`, `UpdateView`, `DeleteView`	преподаватель
`/submissions/`	`my_submissions`	студент
`/submissions/<pk>/delete/`	`submission_delete`	автор, до проверки
`/review/`, `/submissions/<pk>/grade/`	`review`, `grade_submission`	преподаватель
`/grades/`	`grades_table`	авторизованные
`/profile/`	`profile`	авторизованные
`/admin/`	Django Admin	преподаватель

4.4 Регистрация и вход

При регистрации студент указывает ФИО и группу, после чего выполняется автоматический вход (рисунки 8 и 9).

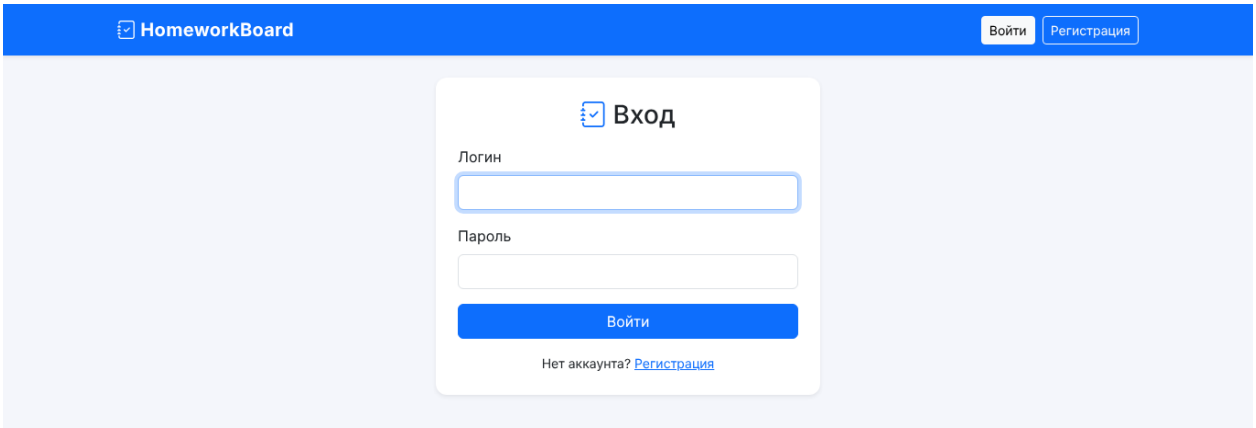


Рисунок 8 — Страница входа

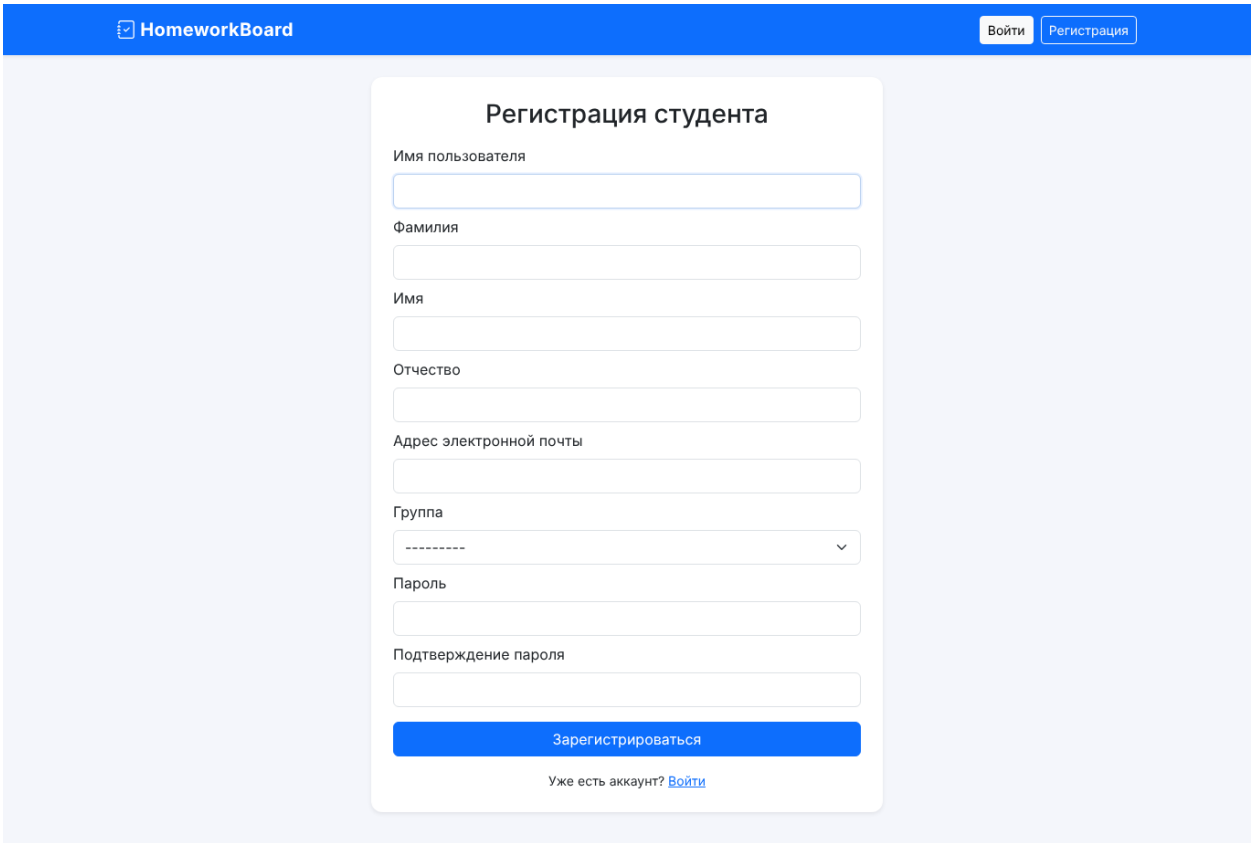


Рисунок 9 — Регистрация студента

4.5 Список домашних заданий

Задания выводятся карточками с предметом, сроком (бейдж с числом оставшихся дней или отметкой «срок истёк»), преподавателем и статусом сдачи. Поиск работает по теме и тексту задания. Фильтры: предмет, срок (актуальные или просроченные) и статус студента (не сдано, на проверке, оценено). Пагинация сохраняет параметры фильтрации благодаря тегу `{% querystring %}`.

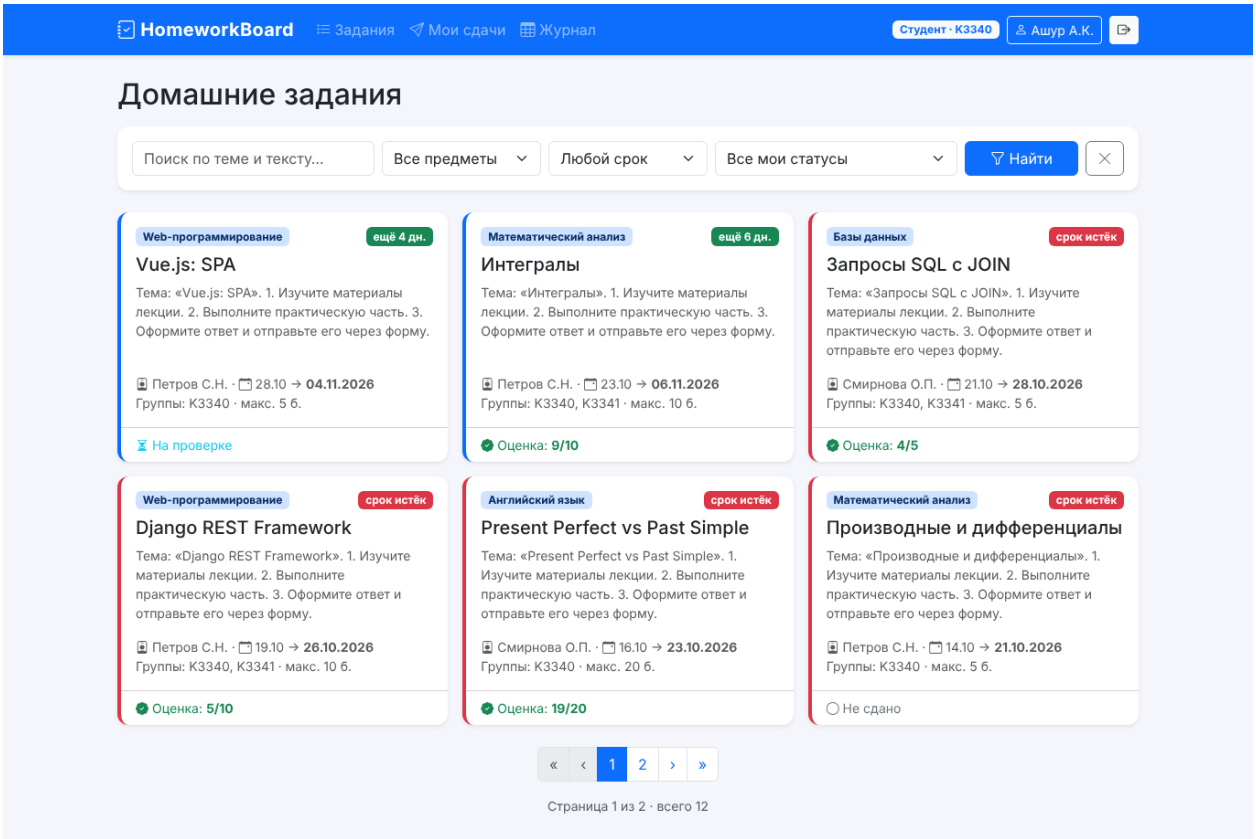


Рисунок 10 — Список заданий глазами студента

Преподаватель видит задания всех групп, число сдач и количество работ, ожидающих проверки. Ему доступны дополнительные фильтры «группа» и «только мои задания», а также кнопка «Выдать задание» (рисунок 11).

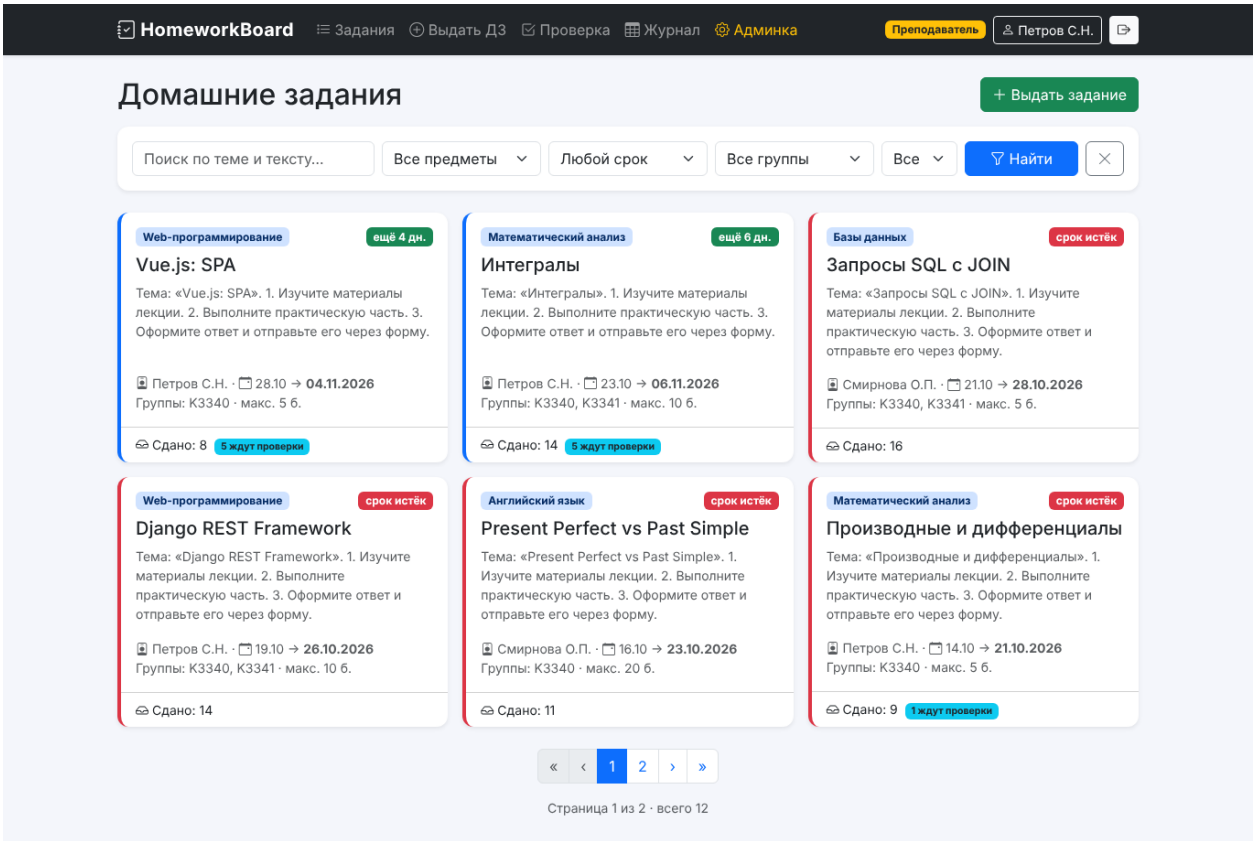


Рисунок 11 — Список заданий глазами преподавателя

4.6 Сдача работы

На странице задания студент видит текст, период выполнения, дедлайн и информацию о штрафах. Справа расположена форма сдачи. До проверки ответ можно изменить или отменить; после проверки выводятся оценка и комментарий преподавателя (рисунки 12 и 13).

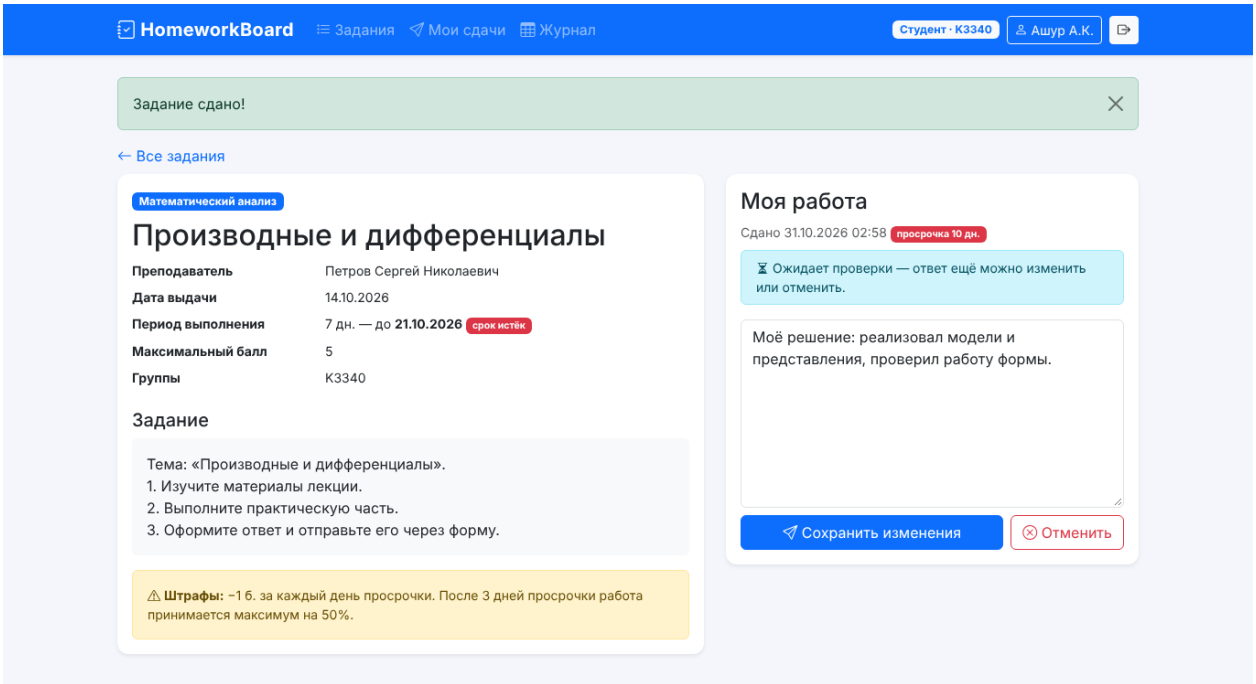


Рисунок 12 — Сданная работа, ожидающая проверки

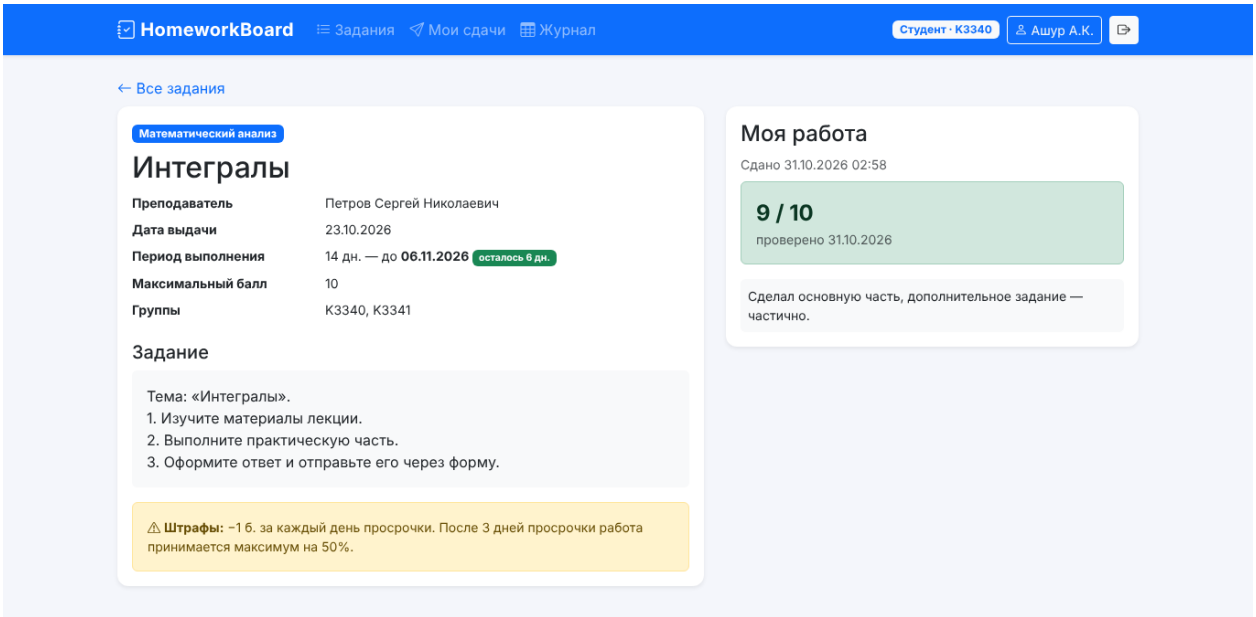


Рисунок 13 — Проверенная работа с оценкой

Раздел «Мои сдачи» содержит таблицу работ студента с фильтром по предмету и статусу, отметкой о просрочке и оценками (рисунок 14).

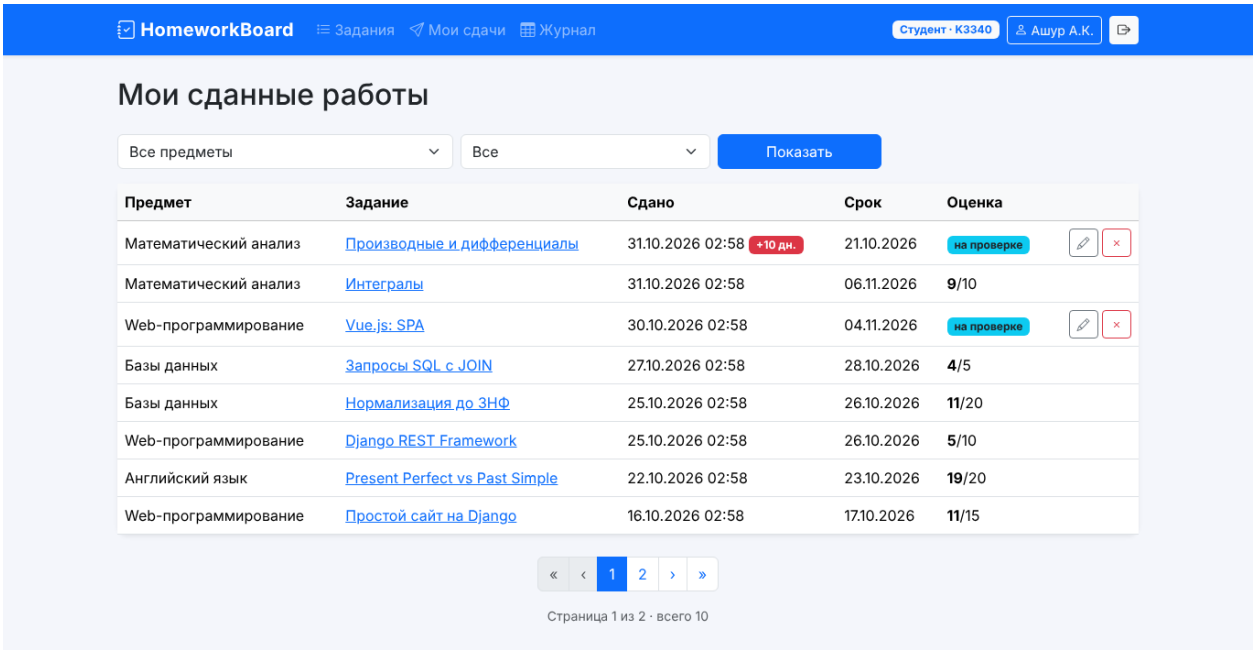


Рисунок 14 — Раздел «Мои сдачи»

4.7 Журнал оценок

Журнал — таблица «студенты × задания» для группы. Цвет ячейки зависит от процента набранных баллов, в конце строки выводятся сумма и индикатор выполнения. Доступны фильтр по предмету и поиск по фамилии; преподаватель дополнительно выбирает группу. Строка текущего студента подсвечивается (рисунки 15 и 16).

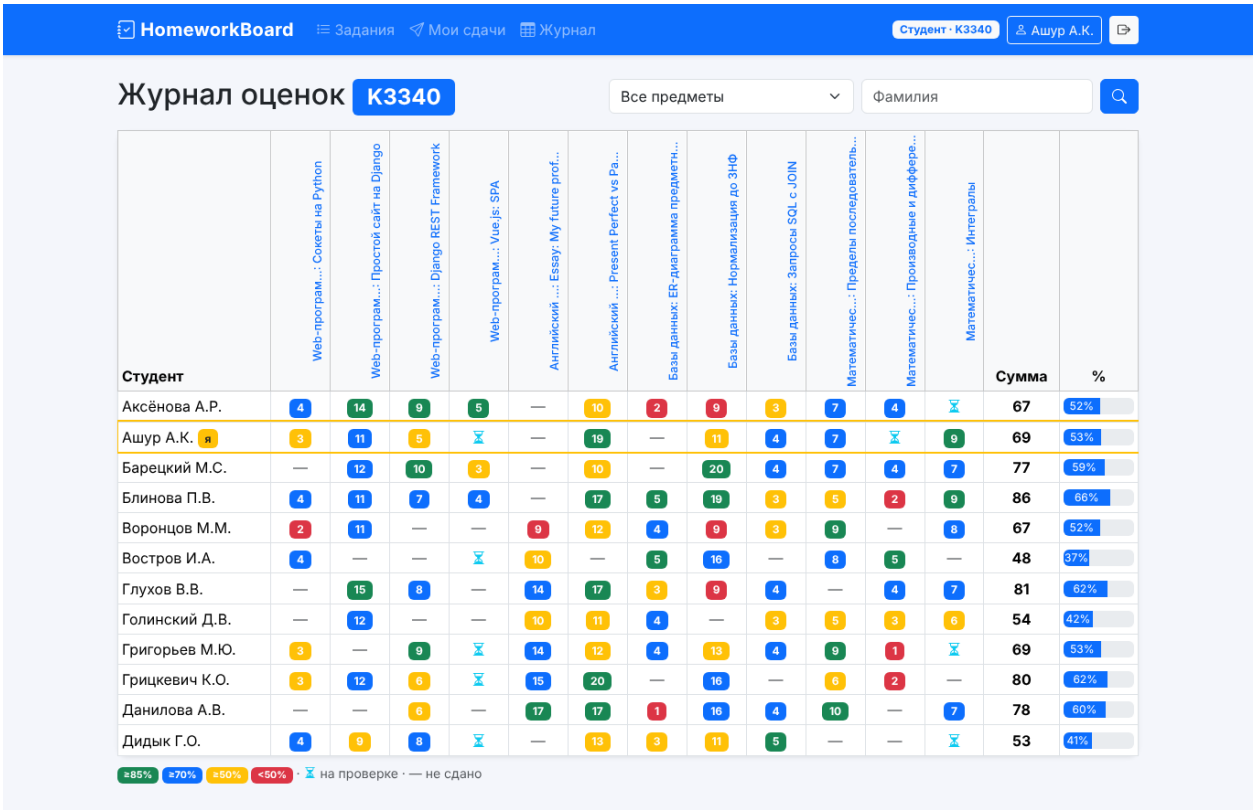


Рисунок 15 — Журнал оценок группы (студент)

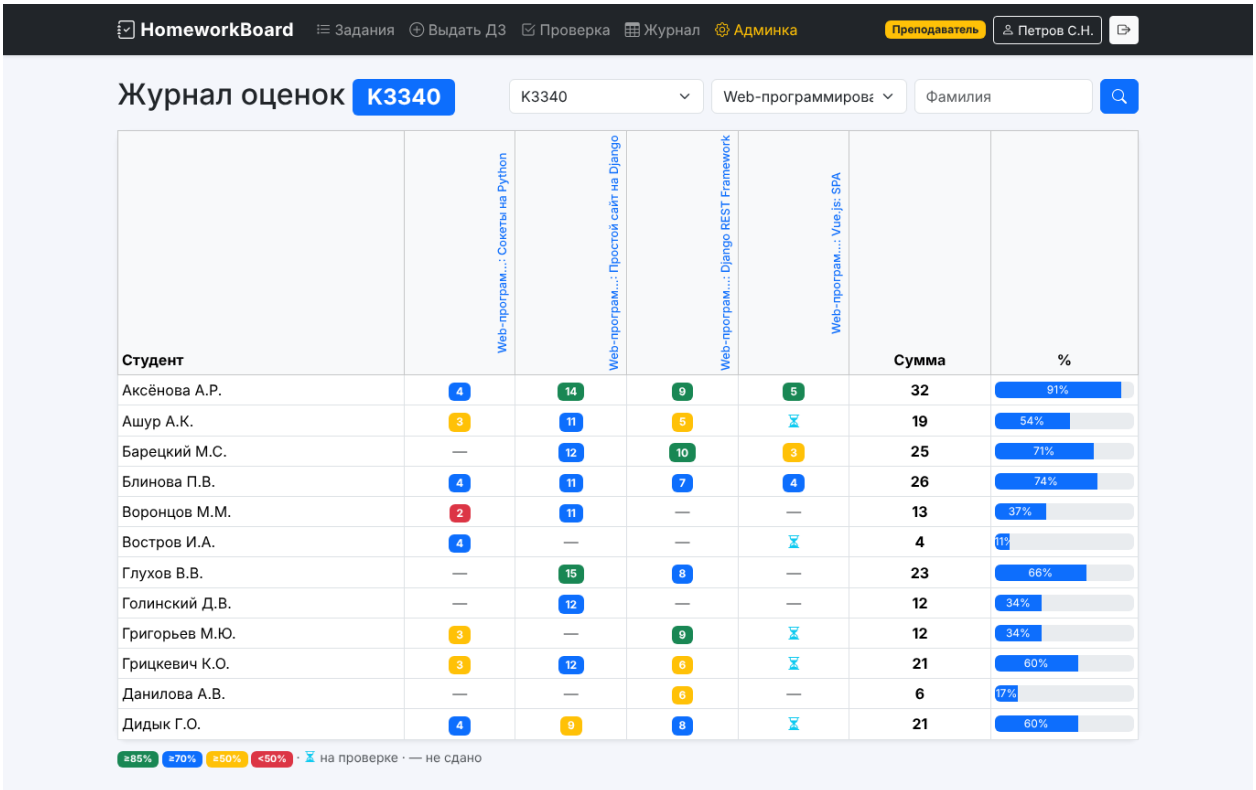


Рисунок 16 — Журнал оценок с фильтром по предмету (преподаватель)

4.8 Интерфейсы преподавателя

Выдача задания выполняется через форму `ModelForm` с выбором групп, периода выполнения, максимального балла и штрафов (рисунок 17).

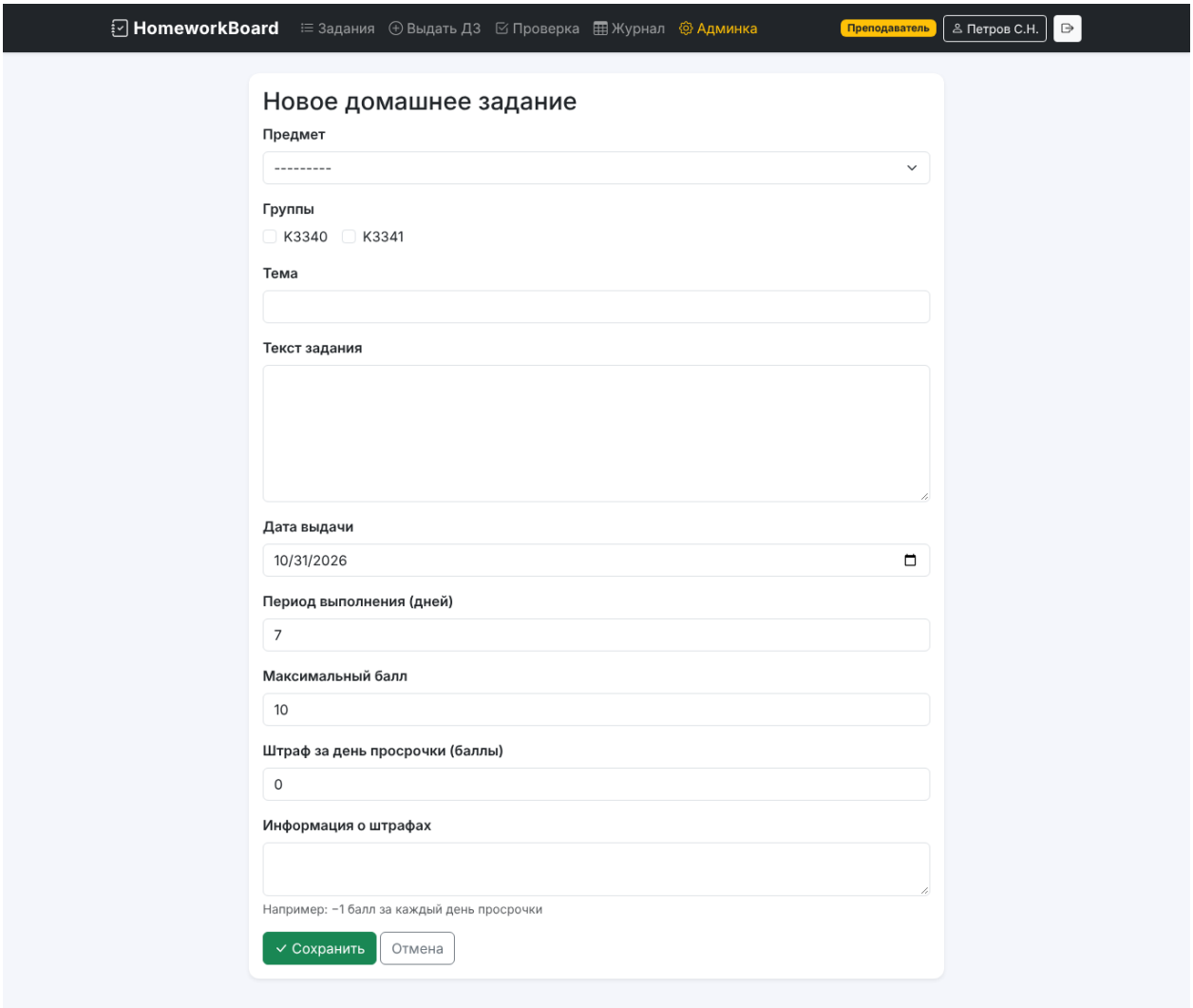


Рисунок 17 — Форма выдачи домашнего задания

Очередь проверки содержит вкладки «ждут проверки», «проверенные», «все» и фильтры по группе, предмету и фамилии. Для каждой работы показаны просрочка и рекомендуемая оценка с учётом штрафа (рисунок 18). На странице задания преподаватель видит все сданные работы с формой оценки и список студентов, которые ещё не сдали работу (рисунок 19).

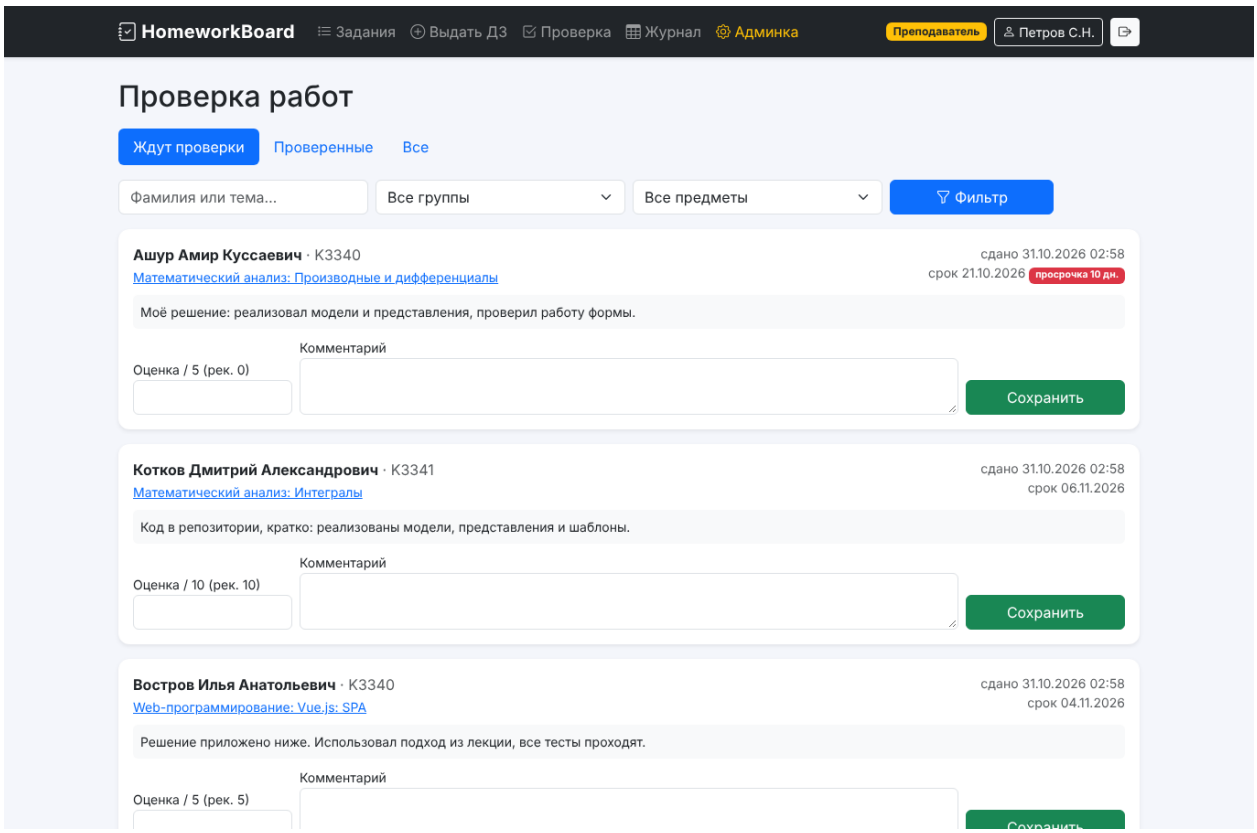


Рисунок 18 — Очередь проверки работ

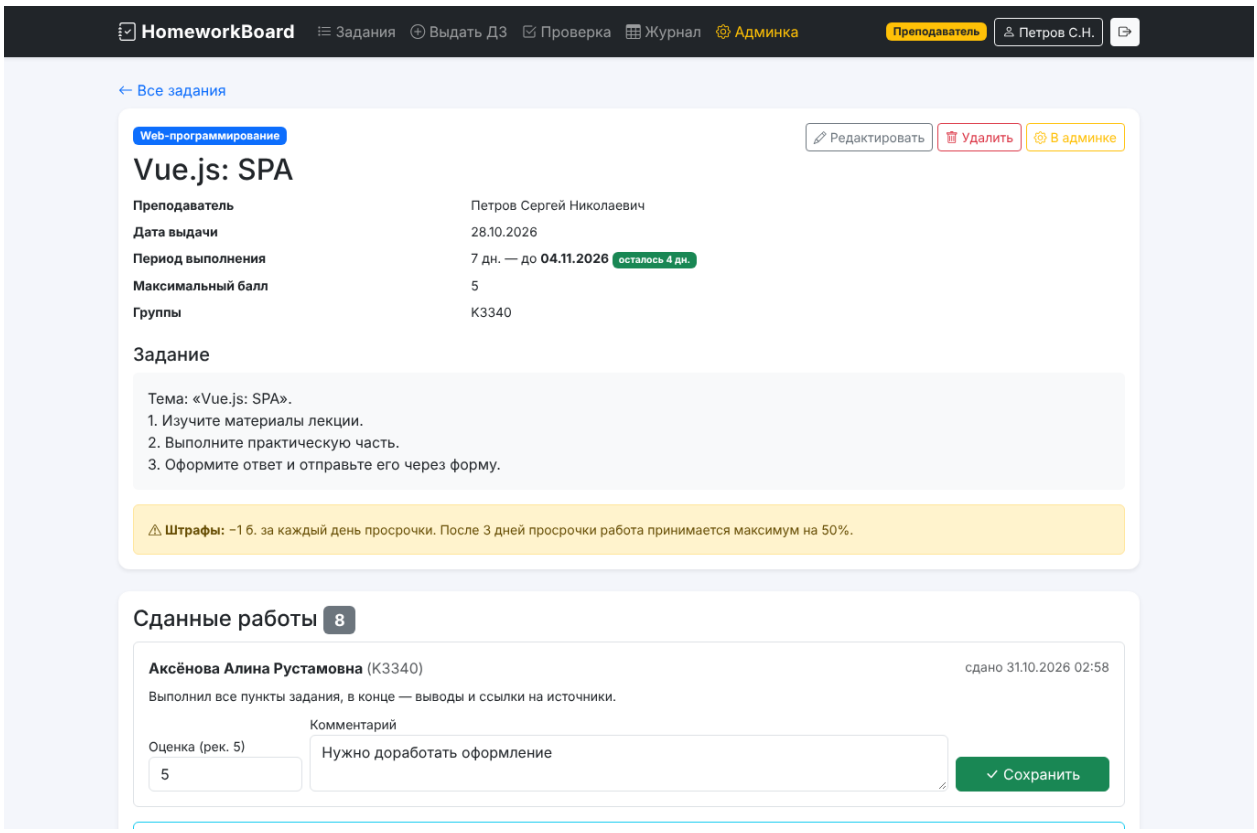


Рисунок 19 — Задание глазами преподавателя

4.9 Django Admin

В админ-панели оценки выставляются прямо в списке сданных работ (`list_editable = ('grade',)`). Доступны фильтры по предмету, группе и наличию оценки, рядом выводятся дни просрочки и рекомендуемая оценка (рисунок 20).

```
@admin.register(Submission)
class SubmissionAdmin(admin.ModelAdmin):
    list_display = ('student', 'homework', 'submitted_at',
                    'days_late', 'suggested_grade', 'grade')
    list_editable = ('grade',)
    list_filter = ('homework__subject', 'student__group',
                   ('grade', admin.EmptyFieldListFilter))
    search_fields = ('student__last_name', 'homework__title', 'text')
```

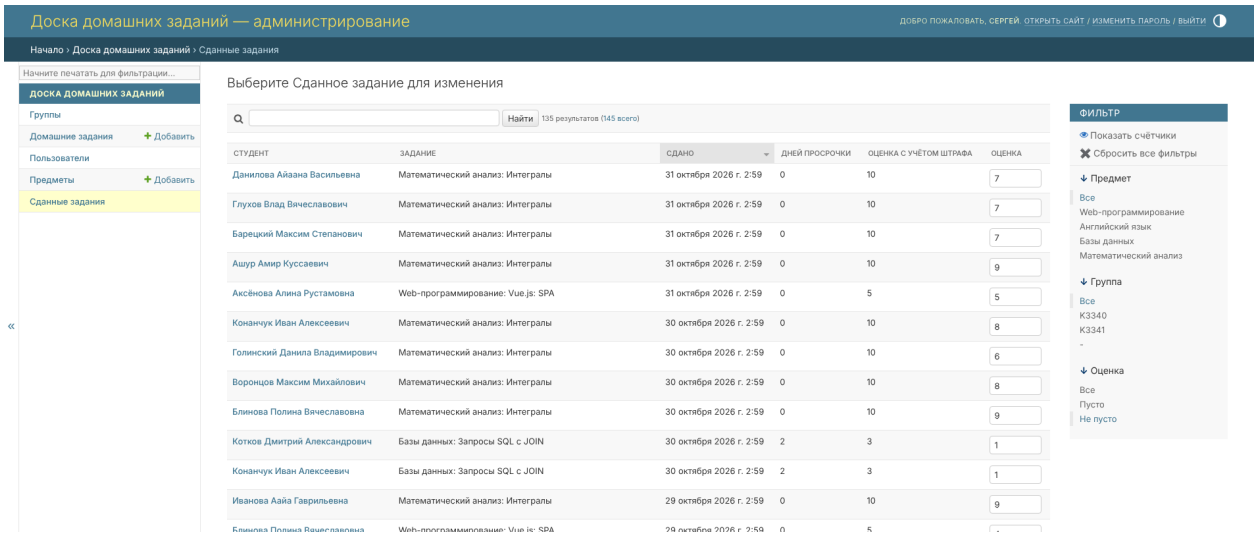


Рисунок 20 — Выставление оценок в Django Admin (вход под преподавателем)

4.10 Тестирование

Для проекта написано 17 автоматических тестов (`board/tests.py`), запуск — командой `python manage.py test board`. Тесты проверяют:

– вычисление дедлайна и штрафа за просрочку;

– регистрацию и перенаправление неавторизованных пользователей;

– запрет доступа студента к страницам преподавателя;

– видимость заданий только своей группы и запрет смены группы в профиле;

– сдачу, изменение и отмену работы, блокировку проверенной работы;

– выставление оценки с проверкой максимума и защиту от перенаправления на сторонний сайт;

– права преподавателя в Django Admin;
– устойчивость фильтров к пустым и неверным параметрам, поиск, пагинацию и журнал.

Все тесты проходят успешно.

4.11 Запуск проекта

Структура репозитория проекта:

```
laboratory_work_2/
├── practice/django_project_ashur/  # практики 2.1–2.3
│   └── project_first_app/
├── homework_board/                 # лабораторная работа
│   ├── homework_board/             # настройки проекта
│   ├── board/                      # приложение и тесты
│   └── static/vendor/              # Bootstrap
├── docs/, mkdocs.yml               # отчёт MkDocs
├── requirements.txt
└── .gitignore
```

1. Установить зависимости: `pip install -r requirements.txt`.
2. Создать базу данных PostgreSQL `homework_board`. Параметры подключения задаются переменными окружения `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`.
3. В папке `homework_board` выполнить `python manage.py migrate`, затем `python manage.py seed` для загрузки демонстрационных данных.
4. Запустить сервер командой `python manage.py runserver` и открыть `http://127.0.0.1:8000`.

Таблица 5 — Демонстрационные учётные записи

Роль	**Логин**	**Пароль**
Суперпользователь	`admin`	`admin`
Преподаватель	`teacher`	`teacher123`
Студент	`ashur`	`student123`

Заключение

В ходе работы выполнены практические работы 2.1–2.3: создан проект Django, описаны модели со связями «один ко многим» и «многие ко многим» через ассоциативную сущность, реализованы функциональные и классовые представления, формы и расширенная модель пользователя на основе `AbstractUser`.

По варианту 2 разработан сайт «Доска домашних заданий» на Django и PostgreSQL. Реализованы регистрация, просмотр заданий со сроками и штрафами, сдача работ в текстовом виде с возможностью изменения и отмены, выставление оценок преподавателем в Django Admin и на сайте, а также журнал оценок группы. Для максимальной оценки выполнены стилизация интерфейса средствами Bootstrap, пагинация, поиск и фильтрация, разграничение прав студента и преподавателя в клиентской части. Работоспособность проверена 17 автоматическими тестами.

В результате получены практические навыки реализации web-сервисов средствами Django.